

Visualization Literacy Analysis

Derek Harter, Shulan Lu

10/19/2025

Read in data files

```
#grab files from google drive (only have to do this once)
# source("getFromGoogleDrive.R")

#get the first 6? characters of each data file
#get unique values of these
# this is list of subject ids

raw_file_names <- list.files("../AccData")
first_six <- substr(raw_file_names, 1, 6)
sub_ids <- unique(first_six)

# fast_RT = data.frame(ParticipantId = character(),
#                       TrialName = character(),
#                       type = character(),
#                       time = numeric()
# )

all_data <- NULL

for (i in 1:length(sub_ids)){

  if (grepl("~$", sub_ids[i], fixed = T)){
    next
  }

  temp_file1 <- read_xlsx(paste0("../AccData/", sub_ids[i], "_1.xlsx")) %>%
    slice(1:17) %>%
    select(-starts_with("Order")) %>%
    rename(correct = 8)

  temp_file2 <- read_xlsx(paste0("../AccData/", sub_ids[i], "_2.xlsx")) %>%
    slice(1:17) %>%
    select(-starts_with("Order")) %>%
    rename(correct = 8)

  new_temp <- temp_file1 %>%
    bind_cols(temp_file2$correct) %>%
    rename(Correct_1 = correct, Correct_2 = "...9") %>%
    mutate(AnswerRT = TimeToBeginInput - TimeToReadQuestion)
```

```

all_data <- all_data %>%
  bind_rows(new_temp)
}

all_data <- all_data %>%
  rename(readRT = TimeToReadQuestion, totalRT = TimeToBeginInput)

#read in trialtype key (I created this from an early version of the previous paper)
trial_type_key <- read.csv("../trial_type_key.csv", stringsAsFactors = F)

all_data <- all_data %>%
  mutate(TrialType = trial_type_key$TrialType[match(TrialName, trial_type_key$TrialName)]) %>%
  mutate(TrialType = paste0("Type", TrialType))

```

Basic checks

```

#how many participants per condition
all_data %>%
  group_by(ParticipantId, Condition) %>%
  summarize(ntrials = n()) %>%
  group_by(Condition) %>%
  summarize(nsubs = n()) %>%
  kable()

```

`summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups` argument.

Condition	nsubs
VR	50
VR Monitor	39
VR Monitor Stereo	33

#why is the balance so off?

Remove outliers based on RT

```

#removing on trial-by-trial basis

#remove answerRTs below 2000ms first
all_data_remove <- all_data %>%
  filter(AnswerRT >= 2000)
dim(all_data)[1]

```

```
## [1] 2074
```

```
dim(all_data_remove)[1]
```

```
## [1] 2067
```

```
#drops 7 trials
```

```
rt_data_summary <- all_data %>%  
  group_by(TrialName) %>%  
  summarize(meanAnswerRT = mean(AnswerRT, na.rm = T),  
            sdAnswerRT = sd(AnswerRT, na.rm = T),  
            UB = meanAnswerRT + 3*sdAnswerRT,  
            LB = meanAnswerRT - 3*sdAnswerRT)  
rt_data_summary
```

```
## # A tibble: 17 x 5  
##   TrialName      meanAnswerRT sdAnswerRT      UB      LB  
##   <chr>          <dbl>      <dbl>    <dbl>  <dbl>  
## 1 BarChartQ1      27611.    14762.   71897. -16675.  
## 2 BarChartQ2      16921.    13338.   56935. -23093.  
## 3 BarChartQ3      15609.     9948.  45452. -14235.  
## 4 BarChartQ4      10288.     8978.  37222. -16646.  
## 5 LineChartQ1      49185.   29505. 137699. -39329.  
## 6 LineChartQ2      40127.   31660. 135107. -54854.  
## 7 LineChartQ3      27190.   17504.  79701. -25322.  
## 8 LineChartQ4      14779.   16568.  64483. -34925.  
## 9 LineChartQ5      27877.   17250.  79628. -23874.  
## 10 ScatterplotQ1    32801.   24294. 105682. -40080.  
## 11 ScatterplotQ2    29636.   23980. 101576. -42304.  
## 12 ScatterplotQ3    45580.   33014. 144623. -53463.  
## 13 ScatterplotQ4    35987.   25402. 112194. -40219.  
## 14 ScatterplotQ5    59805.   42684. 187859. -68248.  
## 15 SurfacePlotQ1    56608.   36042. 164733. -51516.  
## 16 SurfacePlotQ2    65110.   50062. 215297. -85076.  
## 17 SurfacePlotQ3    48373.   37424. 160646. -63900.
```

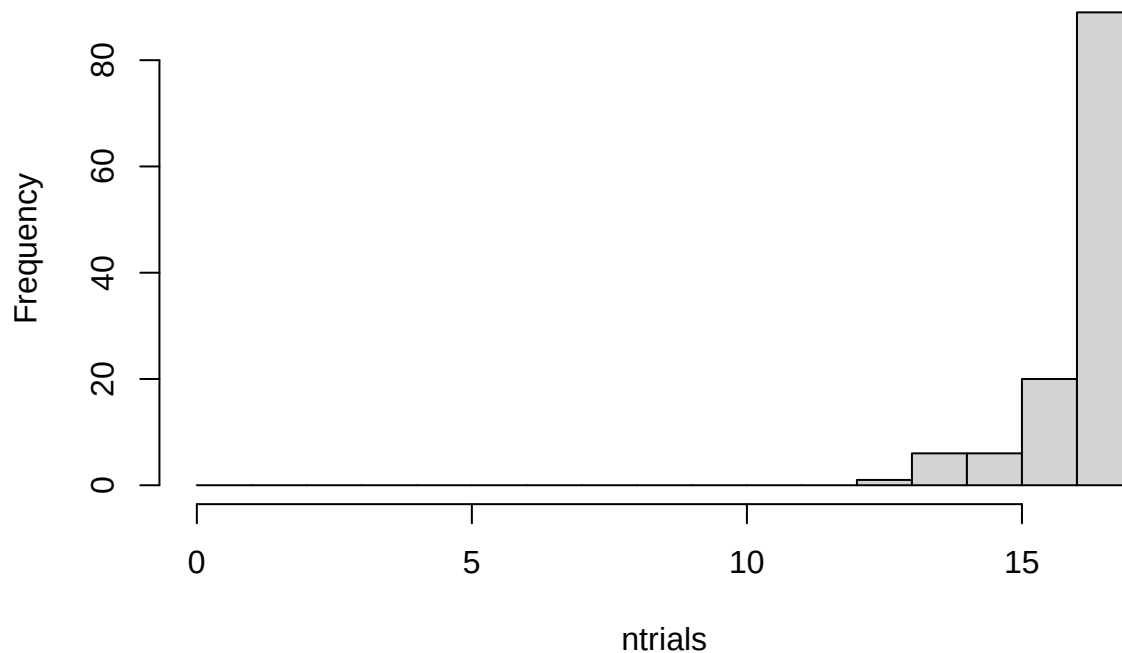
```
all_data_no_outliers <- all_data_remove %>%  
  group_by(TrialName) %>%  
  filter(!(abs(AnswerRT - mean(AnswerRT)) > 3.0*sd(AnswerRT)))  
dim(all_data_no_outliers)[1]
```

```
## [1] 2020
```

```
#drops 47 more trials
```

```
all_data_no_outliers %>%  
  group_by(ParticipantId) %>%  
  summarize(ntrials = n()) %>%  
  with(hist(ntrials, breaks = 0:17))
```

Histogram of ntrials



##maybe consider replacing outliers with means instead of removing them?

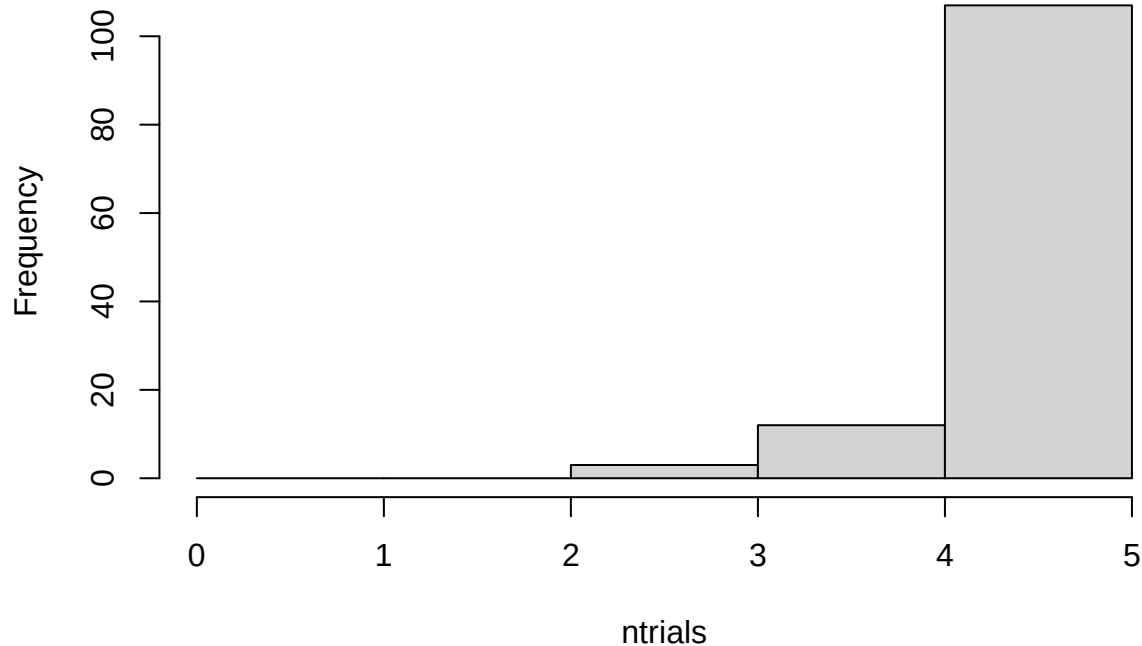
Extract Only One Type of Chart

Create a dataframe from the data with no outliers that contains only XChartQ1 through XChartQ4 trials for the following analysis.

```
all_data_no_outliers <- all_data_no_outliers %>%  
  filter(grepl("ScatterplotQ[0-9]", TrialName))
```

```
all_data_no_outliers %>%  
  group_by(ParticipantId) %>%  
  summarize(ntrials = n()) %>%  
  with(hist(ntrials, breaks = 0:5))
```

Histogram of ntrials



Compare RTs for conditions and question type (three types: identify, relate, predict)

```
#compare read times (should be no differences of condition)  
#compare answer times (potentially a difference)
```

```
trial_type_means <- all_data_no_outliers %>%  
  group_by(ParticipantId, Condition, TrialType) %>%  
  summarize(mean_readRT = mean(readRT),  
            mean_answerRT = mean(AnswerRT),  
            n = n())
```

```
## `summarise()` has grouped output by 'ParticipantId', 'Condition'. You can override using  
## the `.groups` argument.
```

```
# first, make .csv files in wide format to double check in statview  
read_rt_type_wider <- trial_type_means %>%  
  select(ParticipantId, Condition, TrialType, mean_readRT) %>%  
  pivot_wider(names_from = TrialType, values_from = mean_readRT)  
answer_rt_type_wider <- trial_type_means %>%  
  select(ParticipantId, Condition, TrialType, mean_answerRT) %>%  
  pivot_wider(names_from = TrialType, values_from = mean_answerRT)  
write.csv(read_rt_type_wider, file = "readtypeRTs.csv", row.names = F)  
write.csv(answer_rt_type_wider, file = "answertypeRTs.csv", row.names = F)
```

```
#### READ RTs ####
```

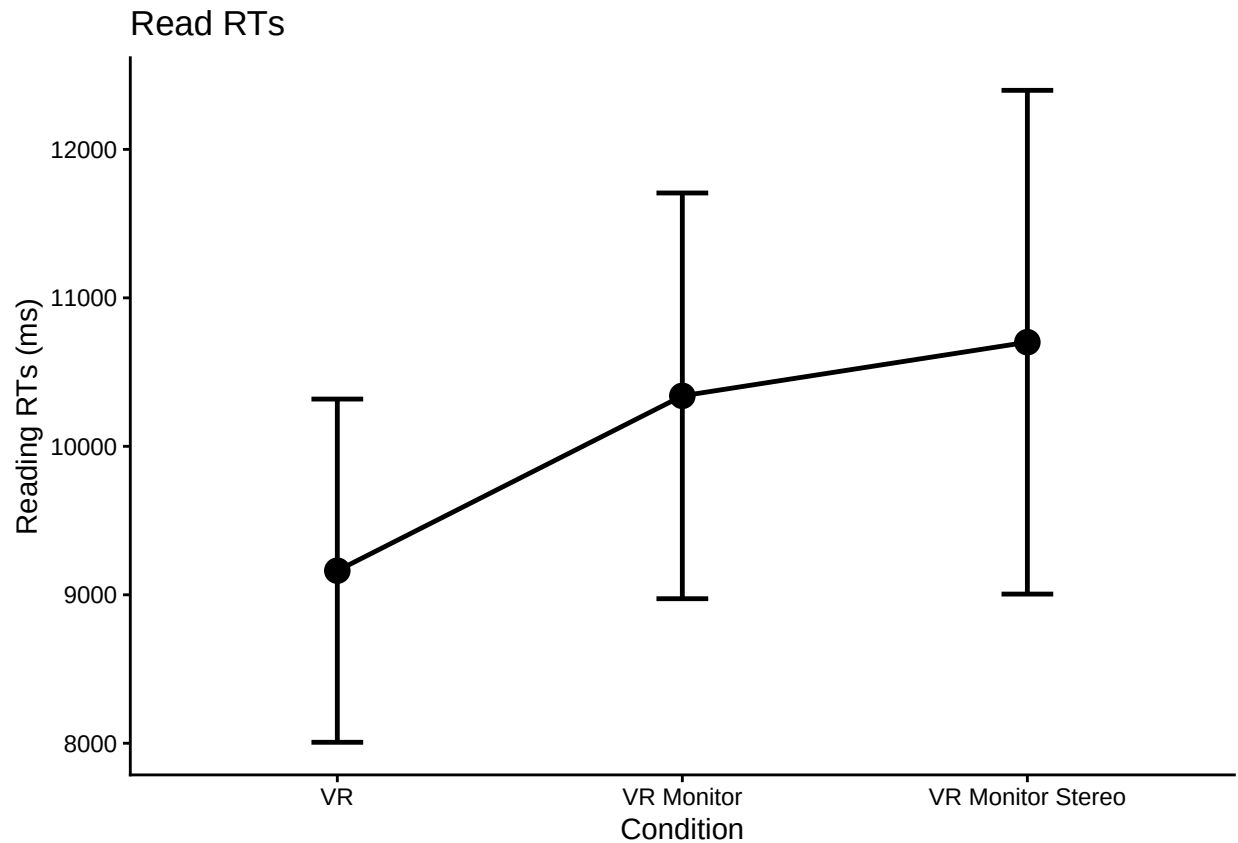
```
#make some plots
```

```
#just condition main effect
```

```
readplot1 <- all_data_no_outliers %>%  
  group_by(ParticipantId, Condition) %>%  
  summarize(overallmean = mean(readRT)) %>%  
  group_by(Condition) %>%  
  summarize(overall_condition_mean = mean(overallmean),  
            se = std.error(overallmean),  
            n = n(),  
            CI = qt(0.975,df=n-1)*se)
```

```
## `summarise()` has grouped output by 'ParticipantId'. You can override using the ``.groups`  
## argument.
```

```
ggplot(readplot1, aes(Condition,  
                      overall_condition_mean,  
                      group = 1,  
                      ymin = overall_condition_mean - CI,  
                      ymax = overall_condition_mean + CI)) +  
  theme_classic() +  
  geom_point(size = 4) +  
  geom_errorbar(width = .15, linewidth = 0.85) +  
  geom_line(linewidth = 0.85) +  
  labs(y = "Reading RTs (ms)", title = "Read RTs")
```



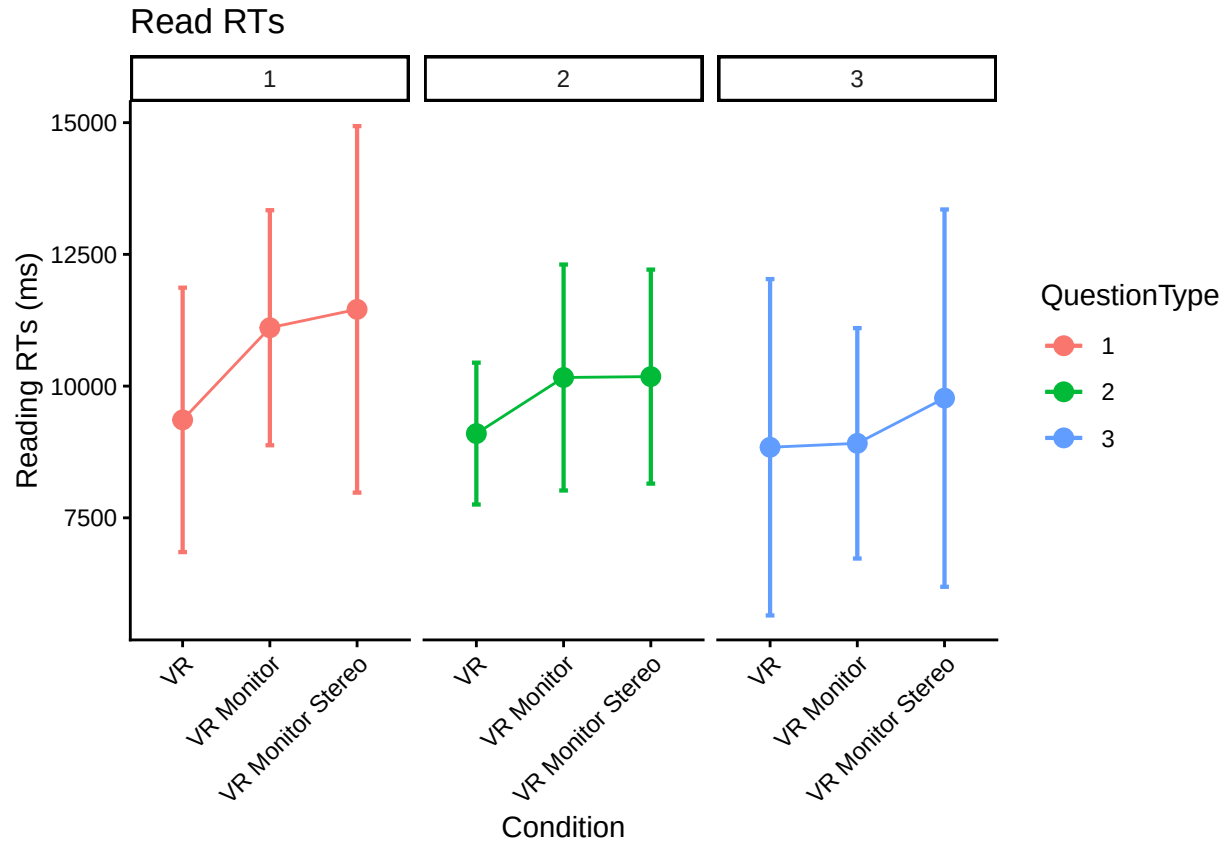
#make a little plot using wide format

```
superbPlot(read_rt_type_wider,
  BSFactors = "Condition",
  WSFactors = "QuestionType(3)",
  variables = c("Type1", "Type2", "Type3"),
  statistic = "meanNArm",
  errorbar = "CI",
  gamma = 0.95,
  adjustments = list(
    purpose = "difference"
  ),
  plotLayout = "line",
  factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1)) +
facet_wrap(vars(QuestionType)) +
labs(y = "Reading RTs (ms)", title = "Read RTs")
```

superb::WARNING(2): There are missing data in your dependent variable(s).

superb may show empty summaries... consult help(measuresWithMissingData)

superb::ADVICE: Some of the groups' variances are heterogeneous. Consider using purpose="tryon".



```
read_rt_type_anova <- aov_ez(id = "ParticipantId",
  dv = "mean_readRT",
  data = trial_type_means,
  within = "TrialType",
  between = "Condition",
  anova_table = list(es = "pes") #might want to double-check these
)
```

```
## Converting to factor: Condition
```

```
## Warning: Missing values for 4 ID(s), which were removed before analysis:
```

```
## AR9449, HA1682, HA3649, OI0406
```

```
## Below the first few rows (in wide format) of the removed cases with missing data.
```

```
##      ParticipantId      Condition      Type1      Type2 Type3
## # 34      AR9449              VR  8601.852  8690.124   NA
## # 56      HA1682 VR Monitor Stereo  9272.346 21390.610   NA
## # 57      HA3649      VR Monitor 15866.913  8455.628   NA
## # 92      OI0406 VR Monitor Stereo  6026.285  7138.086   NA
```

```
## Contrasts set to contr.sum for the following variables: Condition
```

```
kable(nice(read_rt_type_anova))
```

Effect	df	MSE	F	pes	p.value
Condition	2, 115	58389765.88	1.00	.017	.369
TrialType	1.53, 176.52	25780333.24	3.36 *	.028	.050
Condition:TrialType	3.07, 176.52	25780333.24	0.57	.010	.637


```

#posthoc test for trial type
pairs(emmeans(read_rt_type_anova, "TrialType"), adjust = "Tukey")

## contrast      estimate SE  df t.ratio p.value
## Type1 - Type2      942 453 115   2.079  0.0986
## Type1 - Type3     1511 727 115   2.078  0.0990
## Type2 - Type3      569 554 115   1.027  0.5616
##
## Results are averaged over the levels of: Condition
## P value adjustment: tukey method for comparing a family of 3 estimates

#### ANSWER RTs ####

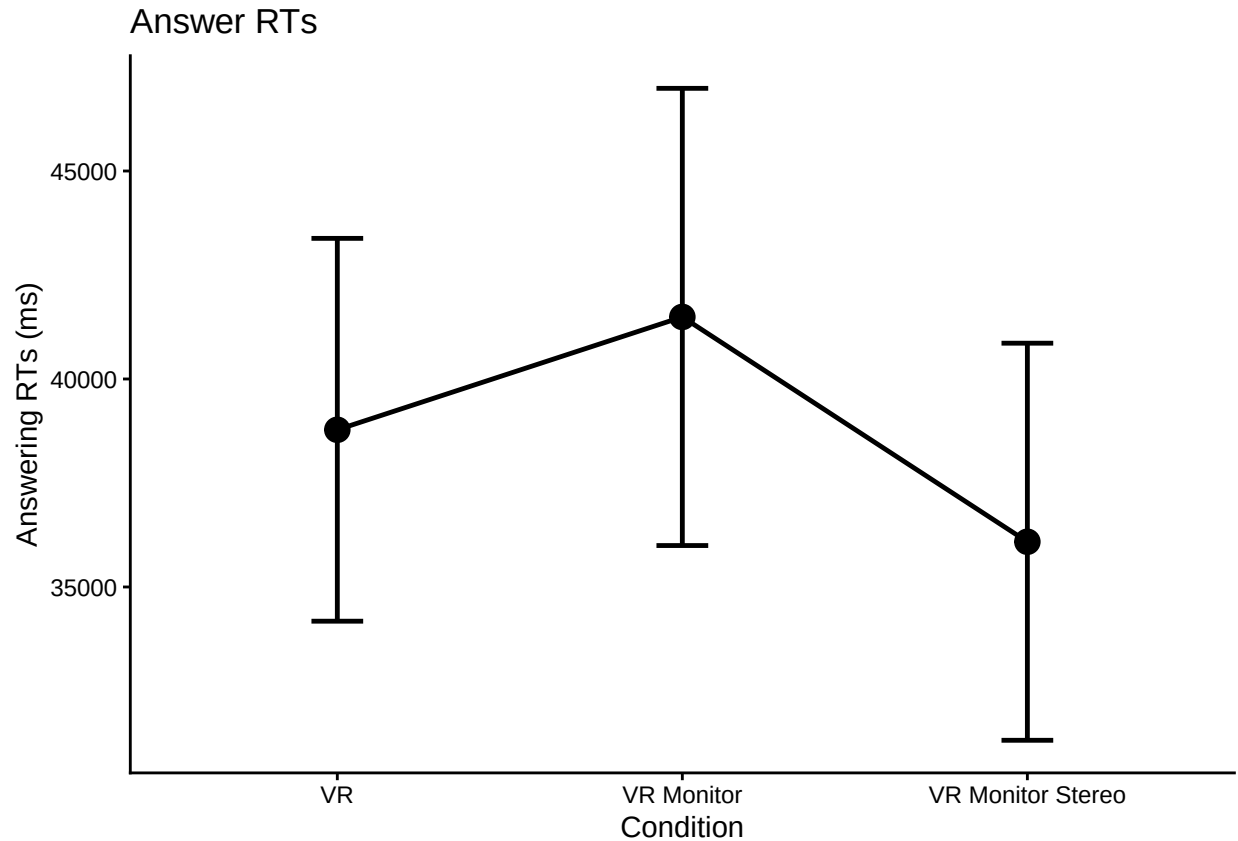
#make some plots

#just condition main effect
answerplot1 <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition) %>%
  summarize(overallmean = mean(AnswerRT)) %>%
  group_by(Condition) %>%
  summarize(overall_condition_mean = mean(overallmean),
            se = std.error(overallmean),
            n = n(),
            CI = qt(0.975,df=n-1)*se)

## `summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups`
## argument.

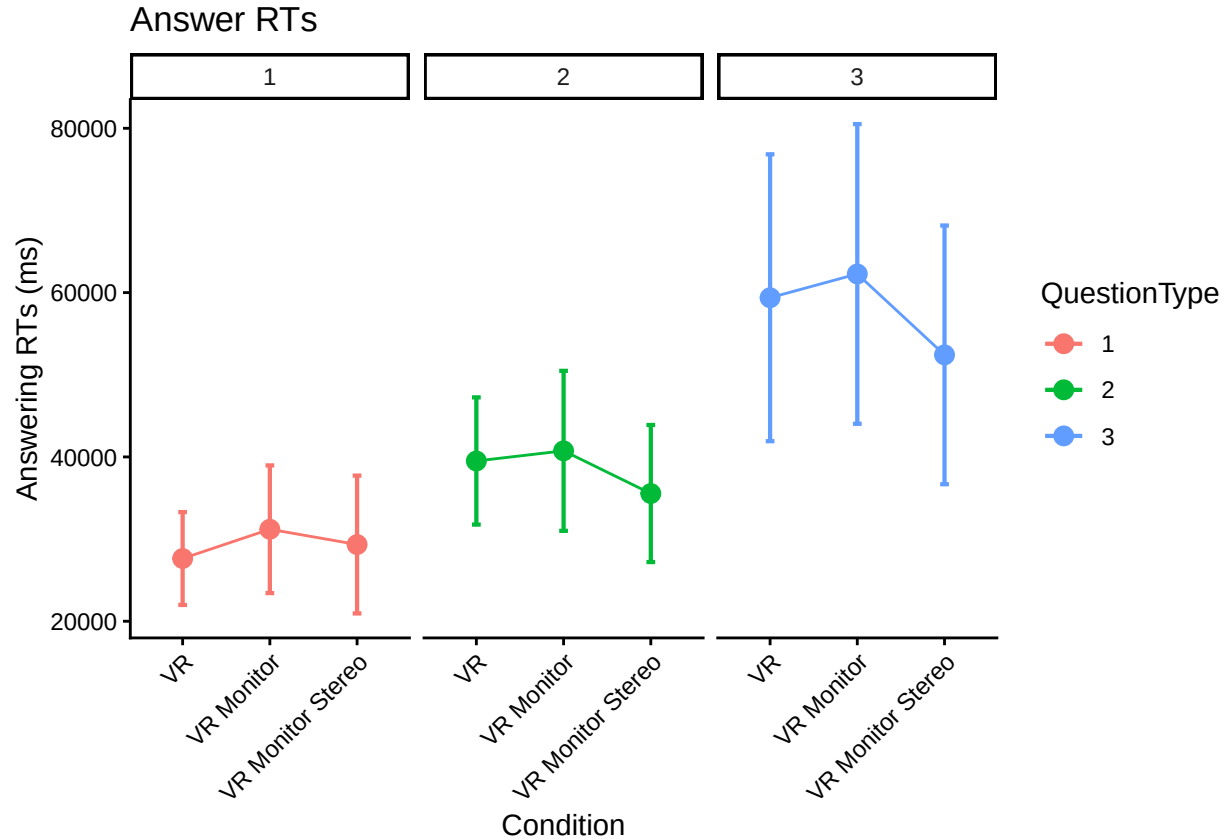
ggplot(answerplot1, aes(Condition,
                        overall_condition_mean,
                        group = 1,
                        ymin = overall_condition_mean - CI,
                        ymax = overall_condition_mean + CI)) +
  theme_classic() +
  geom_point(size = 4) +
  geom_errorbar(width = .15, linewidth = 0.85) +
  geom_line(linewidth = 0.85) +
  labs(y = "Answering RTs (ms)", title = "Answer RTs")

```



```
#make the little plot
superbPlot(answer_rt_type_wider,
  BSFactors = "Condition",
  WSFactors = "QuestionType(3)",
  variables = c("Type1", "Type2", "Type3"),
  statistic = "meanNArm",
  errorbar = "CI",
  gamma = 0.95,
  adjustments = list(
    purpose = "difference"
  ),
  plotLayout = "line",
  factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1))+
facet_wrap(vars(QuestionType))+
labs(y = "Answering RTs (ms)", title = "Answer RTs")
```

```
## superb::WARNING(2): There are missing data in your dependent variable(s).
## superb may show empty summaries... consult help(measuresWithMissingData)
```



```
answer_rt_type_anova <- aov_ez(id = "ParticipantId",
  dv = "mean_answerRT",
  data = trial_type_means,
  within = "TrialType",
  between = "Condition",
  anova_table = list(es = "pes") #might want to double-check these
)
```

```
## Converting to factor: Condition
```

```
## Warning: Missing values for 4 ID(s), which were removed before analysis:
```

```
## AR9449, HA1682, HA3649, OI0406
```

```
## Below the first few rows (in wide format) of the removed cases with missing data.
```

```
##      ParticipantId      Condition  Type1  Type2 Type3
## # 34      AR9449              VR 26359.88 25601.40   NA
## # 56      HA1682 VR Monitor Stereo 67351.92 36078.05   NA
## # 57      HA3649      VR Monitor 88318.66 52776.73   NA
## # 92      OI0406 VR Monitor Stereo 18782.14 12549.03   NA
```

```
## Contrasts set to contr.sum for the following variables: Condition
```

```
kable(nice(answer_rt_type_anova))
```

Effect	df	MSE	F	pes	p.value
Condition	2, 115	984468368.91	0.68	.012	.509
TrialType	1.45, 166.81	767113326.54	45.70 ***	.284	<.001
Condition:TrialType	2.90, 166.81	767113326.54	0.36	.006	.772

```

#posthoc test for trial type
pairs(emmeans(answer_rt_type_anova, "TrialType"), adjust = "Tukey")

## contrast      estimate    SE df t.ratio p.value
## Type1 - Type2   -10227 1940 115  -5.267 <.0001
## Type1 - Type3   -29422 3620 115  -8.121 <.0001
## Type2 - Type3   -19195 3520 115  -5.451 <.0001
##
## Results are averaged over the levels of: Condition
## P value adjustment: tukey method for comparing a family of 3 estimates

#Question Type 3 much faster than Type 1/Type 2 (this is answering times)

ref <- emmeans(answer_rt_type_anova, ~Condition|TrialType)

pairs(ref, adjust = "Tukey")

## TrialType = Type1:
## contrast      estimate    SE df t.ratio p.value
## VR - VR Monitor      -2035 3160 115  -0.643 0.7965
## VR - VR Monitor Stereo  -791 3360 115  -0.236 0.9699
## VR Monitor - VR Monitor Stereo 1244 3540 115  0.351 0.9343
##
## TrialType = Type2:
## contrast      estimate    SE df t.ratio p.value
## VR - VR Monitor      -635 4190 115  -0.152 0.9874
## VR - VR Monitor Stereo 3519 4450 115  0.790 0.7096
## VR Monitor - VR Monitor Stereo 4154 4690 115  0.885 0.6509
##
## TrialType = Type3:
## contrast      estimate    SE df t.ratio p.value
## VR - VR Monitor     -2895 8390 115  -0.345 0.9365
## VR - VR Monitor Stereo 6955 8910 115  0.781 0.7156
## VR Monitor - VR Monitor Stereo 9851 9390 115  1.049 0.5479
##
## P value adjustment: tukey method for comparing a family of 3 estimates

#plot and interaction suggests that the conditions have different effects for different
#question types. posthoc tests indicate a difference between VR and VRMonitor for QType 1

```

Accuracy

```

# do interrater reliability: Fits each trial as independent with two raters
# That is, this ignores participants (complete pooling, which may be problematic)
# 1410 trials from N = 85 participants, not exactly 17 trials per participant
# b/c of excluded trials
irr::icc(select(ungroup(all_data_no_outliers), Correct_1, Correct_2))

## Single Score Intraclass Correlation
##
## Model: oneway
## Type : consistency
##
## Subjects = 592
## Raters = 2

```

```

##      ICC(1) = 0.954
##
## F-Test, H0: r0 = 0 ; H1: r0 > 0
## F(591,592) = 42.1 , p = 5.41e-311
##
## 95%-Confidence Interval for ICC Population Values:
## 0.946 < ICC < 0.96

head(all_data_no_outliers)

## # A tibble: 6 x 11
## # Groups:   TrialName [5]
##   ParticipantId Condition TrialNumber TrialName readRT totalRT UserInput Correct_1
##   <chr>          <chr>      <dbl> <chr>      <dbl>    <dbl> <chr>      <dbl>
## 1 AA0176        VR Monitor      6 ScatterplotQ1 12366. 27154. High income      1
## 2 AA0176        VR Monitor      7 ScatterplotQ2 11326. 23702. Low income       0
## 3 AA0176        VR Monitor      8 ScatterplotQ3 9830. 93848. Positive        0
## 4 AA0176        VR Monitor      9 ScatterplotQ4 19927. 94318. Decreasing      1
## 5 AA0176        VR Monitor     10 ScatterplotQ5 8245. 86052. Try cutting~    1
## 6 AA0876        VR          13 ScatterplotQ1 2435. 40378. 2016.0        0
## # i 3 more variables: Correct_2 <dbl>, AnswerRT <dbl>, TrialType <chr>

# account for non-independence?
#Data has to be narrow
#https://cran.r-project.org/web/packages/cccrm/index.html
icc.dat.narrow <- all_data_no_outliers %>%
  pivot_longer(cols = c('Correct_1', 'Correct_2'))

colnames(icc.dat.narrow)[10] <- "Rater"
icc.dat.narrow$Rater <- as.factor(icc.dat.narrow$Rater)

#Longitudinal concordance for two raters
#By all 17 trials (by items)
# icc.rm.trial.name <- ccclon(icc.dat.narrow,
#                             "value", "ParticipantId", "TrialName", "Rater")
# icc.rm.trial.name
# summary(icc.rm.trial.name)

#By time using trial numbers (by time)
icc.rm.trial.number <- ccclon(icc.dat.narrow,
                             "value", "ParticipantId", "TrialNumber", "Rater" )

## Warning: `ccclon()` was deprecated in cccrm 3.0.0.
## i Please use `ccc_vc()` instead.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was generated.

icc.rm.trial.number

## CCC estimated by variance components:
##      CCC    LL CI 95%    UL CI 95%    SE CCC
## 0.949385915 0.940356117 0.957078885 0.004248889

summary(icc.rm.trial.number)

##      Subjects Subjects-Method Subjects-Time      Method      Error
##      0.0127425554      0.0009092681      0.1821563081      0.0004085254      0.0090727415
##

```

```

## CCC estimated by variance components
##      CCC    LL CI 95%    UL CI 95%      SE CCC
## 0.949385915 0.940356117 0.957078885 0.004248889

# A similar approach
# https://peerj.com/articles/9850/
# test.mod <-
#   lcc(data = hue, subject = "Fruit", resp = "H_mean",
#       method = "Method", time = "Time", qf = 2, qr = 1)
# summary(test.mod)

# icc.dat.narrow$ParticipantId <- as.factor(icc.dat.narrow$ParticipantId)
# icc.dat.narrow$Rater <- as.factor(icc.dat.narrow$Rater)
#
#
# icc.rm.another <-
# lcc(data = icc.dat.narrow,
#     subject = "ParticipantId",
#     resp = "value",
#     method = "Rater",
#     time = "TrialNumber",
#     qf = 1, #polynomial trends (1 to 3)
#     qr = 0, #random effects (0 is random int only, default)
#     components = TRUE)
#
# #LCC = Longitudinal Concordance Correlation
# #LPC = Longitudinal Pearson Correlation
# #LA = Longitudinal Accuracy
# icc.rm.another
#
# #Model coefficients near 1 but only the the last one has a correctly
# #estimated model. Maybe because it's more or less flat over TrialNumber?
# lccPlot(icc.rm.another, type = "lpc")
# lccPlot(icc.rm.another, type = "lcc")
# lccPlot(icc.rm.another, type = "la")

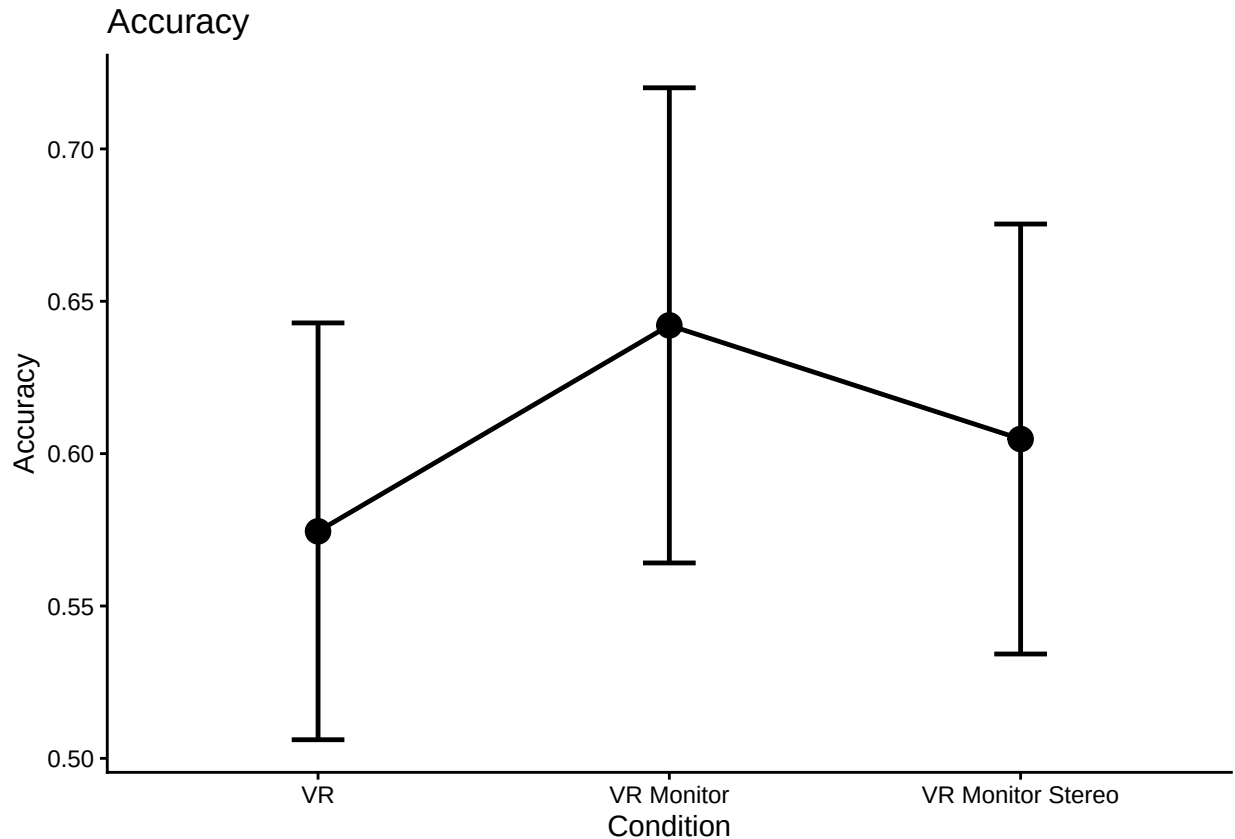
all_data_no_outliers <- all_data_no_outliers %>%
  rowwise() %>%
  mutate(Correct_Avg = mean(c(Correct_1, Correct_2)))

#just condition main effect
accplot1 <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition) %>%
  summarize(overallmean = mean(Correct_Avg)) %>%
  group_by(Condition) %>%
  summarize(overall_condition_mean = mean(overallmean),
    se = std.error(overallmean),
    n = n(),
    CI = qt(0.975,df=n-1)*se)

## `summarise()` has grouped output by 'ParticipantId'. You can override using the `groups`
## argument.

```

```
ggplot(accplot1, aes(Condition,
                     overall_condition_mean,
                     group = 1,
                     ymin = overall_condition_mean - CI,
                     ymax = overall_condition_mean + CI)) +
  theme_classic() +
  geom_point(size = 4) +
  geom_errorbar(width = .15, linewidth = 0.85) +
  geom_line(linewidth = 0.85) +
  labs(y = "Accuracy", title = "Accuracy")
```



```
trial_type_mean_acc <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition, TrialType) %>%
  summarize(mean_acc = mean(Correct_Avg),
            n = n())
```

`summarise()` has grouped output by 'ParticipantId', 'Condition'. You can override using ## the `.groups` argument.

```
acc_type_wider <- trial_type_mean_acc %>%
  select(ParticipantId, Condition, TrialType, mean_acc) %>%
  pivot_wider(names_from = TrialType, values_from = mean_acc)
write.csv(acc_type_wider, file = "typeacc.csv", row.names = F)
```

```
#make the little plot
superbPlot(acc_type_wider,
  BSFactors = "Condition",
```

```

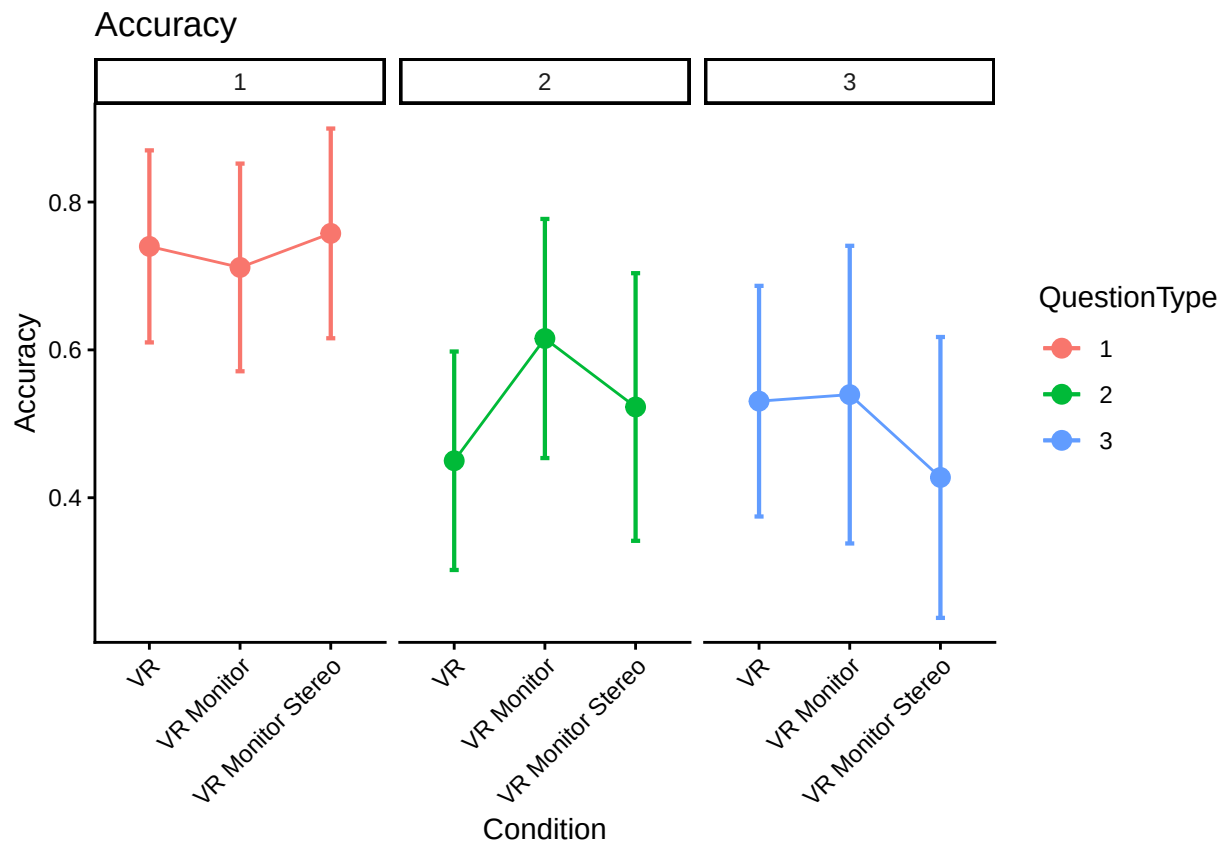
WSFactors = "QuestionType(3)",
variables = c("Type1", "Type2", "Type3"),
statistic = "meanNArm",
errorbar = "CI",
gamma = 0.95,
adjustments = list(
  purpose = "difference"
),
plotLayout = "line",
factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1))+
facet_wrap(vars(QuestionType))+
labs(y = "Accuracy", title = "Accuracy")

```

```

## superb::WARNING(2): There are missing data in your dependent variable(s).
## superb may show empty summaries... consult help(measuresWithMissingData)

```



```

acc_type_anova <- aov_ez(id = "ParticipantId",
  dv = "mean_acc",
  data = trial_type_mean_acc,
  within = "TrialType",
  between = "Condition",
  anova_table = list(es = "pes") #might want to double-check these
)

```



```
## Converting to factor: Condition
## Warning: Missing values for 4 ID(s), which were removed before analysis:
## AR9449, HA1682, HA3649, OI0406
## Below the first few rows (in wide format) of the removed cases with missing data.
##      ParticipantId      Condition Type1 Type2 Type3
## # 34      AR9449      VR      0.5 0.00  NA
## # 56      HA1682 VR Monitor Stereo  1.0 0.75  NA
## # 57      HA3649      VR Monitor  1.0 0.00  NA
## # 92      OI0406 VR Monitor Stereo  0.5 1.00  NA

## Contrasts set to contr.sum for the following variables: Condition
```

```
kable(nice(acc_type_anova))
```

Effect	df	MSE	F	pes	p.value
Condition	2, 115	0.17	0.68	.012	.508
TrialType	1.99, 228.79	0.10	18.07 ***	.136	<.001
Condition:TrialType	3.98, 228.79	0.10	1.80	.030	.129

```
#posthoc test for trial type
```

```
pairs(emmeans(acc_type_anova, "TrialType"), adjust = "Tukey")
```

```
## contrast      estimate      SE df t.ratio p.value
## Type1 - Type2  0.2054 0.0436 115  4.709 <.0001
## Type1 - Type3  0.2365 0.0412 115  5.744 <.0001
## Type2 - Type3  0.0311 0.0434 115  0.716 0.7546
##
```

```
## Results are averaged over the levels of: Condition
```

```
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```
#Question Type 3 much faster than Type 1/Type 2 (this is answering times)
```

```
ref <- emmeans(acc_type_anova, ~Condition | TrialType)
```

```
pairs(ref, adjust = "Tukey")
```

```
## TrialType = Type1:
```

```
## contrast      estimate      SE df t.ratio p.value
## VR - VR Monitor      0.04095 0.0668 115  0.613 0.8132
## VR - VR Monitor Stereo -0.01317 0.0709 115 -0.186 0.9812
## VR Monitor - VR Monitor Stereo -0.05412 0.0748 115 -0.724 0.7500
##
```

```
## TrialType = Type2:
```

```
## contrast      estimate      SE df t.ratio p.value
## VR - VR Monitor -0.17240 0.0771 115 -2.236 0.0695
## VR - VR Monitor Stereo -0.04082 0.0819 115 -0.499 0.8721
## VR Monitor - VR Monitor Stereo 0.13158 0.0863 115 1.524 0.2835
##
```

```
## TrialType = Type3:
```

```
## contrast      estimate      SE df t.ratio p.value
## VR - VR Monitor -0.00886 0.0856 115 -0.103 0.9941
## VR - VR Monitor Stereo 0.10319 0.0909 115 1.135 0.4945
## VR Monitor - VR Monitor Stereo 0.11205 0.0959 115 1.169 0.4742
##
```

```
## P value adjustment: tukey method for comparing a family of 3 estimates
```